EDA, Machine learning and Neural Network Models For Natural Language Processing

Samin Haque
Computer Science And engineering
BRAC University
Dhaka, Bangladesh
samin.haque@g.bracu.ac.bd

Khaled Saifullah Karim
Computer Science And engineering
BRAC University
Dhaka, Bangladesh
Khaled.saifullah.karim@g.bracu.ac.bd

Aalavi Mahin Khan
Computer Science And engineering
BRAC University
Dhaka, Bangladesh
aalavi.mahin.khan@g.bracu.ac.bd

***Abstract*—This project focuses on multi-class text classification, utilizing a variety of word representation techniques and machine learning (ML) and neural network (NN) models. The dataset used in this study was provided in a split format, consisting of training and testing sets. The project began with extensive exploratory data analysis (EDA) to uncover insights, followed by the necessary preprocessing steps, including stopword removal and lemmatization. We applied multiple word representation techniques, namely Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), GloVe, and Skip-gram. For model training, we experimented with machine learning models such as Random Forest, Logistic Regression, and Naive Bayes, alongside deep neural network models like SimpleRNN, GRU, LSTM, and their bidirectional variants. Hyperparameters were manually tuned for each model using validation performance. The models were evaluated using metrics such as accuracy, F1-score, confusion matrix, and classification report. A thorough comparative analysis was performed, identifying the best-performing models and providing a detailed examination of the results. The project concludes with key findings, limitations, and suggestions for future improvements in the field of text classification.**

***Index Terms*—Word Representations, Bag of Words, Bidirectional RNN, Hyperparameter Tuning**

I. Introduction

In recent years, the application of Natural Language Processing (NLP) has gained substantial attention in various domains, including healthcare, finance, and social media. Text classification is a core NLP task with widespread use, ranging from sentiment analysis to spam detection. The objective of this project is to conduct a multi-class text classification study, comparing different word representations and machine learning and neural network models to classify text data effectively.

The dataset for this project consists of labeled textual data, split into training and testing sets, which was subjected to thorough preprocessing and exploratory data analysis (EDA). Preprocessing steps such as stopword removal, stemming, and lemmatization were performed to clean the text, preparing it for model training. EDA provided valuable insights into the dataset, including the distribution of classes, word frequency analysis, and text length variations, guiding the subsequent steps in the feature extraction and model training phases.

We explored various word representation techniques, such as Bag of Words, TF-IDF, and more advanced methods like GloVe and Skip-gram embeddings, to convert raw text into a numerical format suitable for machine learning models. The experimental setup involved training multiple ML models, including Random Forest, Logistic Regression, and Naive Bayes, alongside neural network architectures like SimpleRNN, GRU, LSTM, and their bidirectional variants. Hyperparameter tuning was a critical aspect of this project, ensuring the optimization of each model for maximum performance.

Model evaluation was carried out using accuracy, F1-score, confusion matrix, and classification reports, with a detailed comparative analysis of the results. This study aims to identify the best-performing model, examining the strengths and weaknesses of both machine learning and deep learning approaches. The project concludes by discussing the limitations of the current models and suggesting possible future directions for improving text classification systems.

II. Methodology

A. Exploratory Data Analysis

1) Dataset Composition and Class Distribution: The training set comprised 280,003 samples and the test set contained 59,999 samples, each spanning 10 balanced classes such as Entertainment & Music, Health, Sports, and Politics & Government. Class distribution in the training set was approximately uniform, with each class containing around 27,500–28,200 samples (e.g., Entertainment & Music: 28,267, Health: 28,186, Politics & Government: 27,746). The test set maintained near-perfect balance (6,000 samples per class, except Family & Relationships with 5,999). This confirmed that the dataset was designed to minimize label imbalance.

2) Missing Values and Data Integrity: Basic checks revealed missing data primarily in optional fields: 126,776 missing question content entries and 5,006 missing best answer entries in the training set. However, the composite full text field — used as the main input — contained no missing values. Duplicates were absent, ensuring high data integrity.

3) Text Length Distribution: Word count statistics indicated diverse text lengths across fields. On average, question title contained 10.7 words (max: 39), question content 21.6 words (max: 837), and best answer 59 words (max: 1,169). The aggregated full text averaged 91 words (median: 60, max: 1,334), demonstrating wide variability. This distribution

suggested the need for preprocessing strategies to handle long-tail outliers and standardize input sequence lengths.

4) Lexical Diversity: Lexical richness analysis showed that question title had very high diversity (mean 0.97), while question content was lower (mean 0.48) due to many short or empty responses. The full text field maintained a healthy diversity (mean 0.80), confirming that aggregated content provided sufficient lexical variation for effective learning.

5) Noise and Special Features: The dataset contained 25,850 entries with URLs, 2,217 with email addresses, and 960 with raw HTML fragments, as well as heavy use of punctuation (e.g., 533,113 question marks and 208,104 exclamation marks). These features highlighted the informal, user-generated nature of the corpus and indicated potential preprocessing needs for normalization.

6) Class-Specific Vocabulary: Word frequency analysis revealed strong topical signals within categories. For example, Sports was dominated by "world," "cup," and "team"; Politics & Government featured "bush," "president," and "iraq"; Health included "weight," "lose," and "help." These discriminative lexical cues suggested that even simple bag-of-words models could achieve reasonable baseline performance.

B. Preprocessing Techniques

1) Text Normalization: Each entry in the full text field was subjected to standard NLP cleaning procedures. All text was converted to lowercase, and HTML tags, email addresses, and URLs — which EDA revealed as present in 960, 2,217, and 25,850 instances respectively — were stripped. Special characters, redundant punctuation, and numeric-only tokens were removed to reduce noise.

2) Tokenization and Stopword Removal: The corpus was tokenized using the NLTK Punkt tokenizer. Common English stopwords were removed, leveraging the NLTK stopword list of 179 tokens. This reduced vocabulary redundancy while retaining semantically meaningful words.

3) Lemmatization: Tokens were lemmatized using WordNet Lemmatizer, which mapped inflected forms to their canonical lemmas (e.g., running $\rightarrow$ run, better $\rightarrow$ good). Lemmatization normalized morphological variations, reducing sparsity in the feature space and improving generalization.

4) Vocabulary Construction: After cleaning, a Tokenizer from Keras was fit on the training data, building a vocabulary of 91,533 unique tokens across the corpus. Each token was indexed based on frequency, ensuring frequent words received lower indices for efficient embedding lookup. Rare tokens (frequency < 2) were replaced with an <OOV> marker, consolidating noisy low-frequency words.

5) Sequence Transformation and Padding: The tokenized documents were transformed into integer sequences. To standardize input for neural models, sequences were padded to a fixed maximum length of 500 tokens. EDA revealed that the maximum length in the corpus reached 1,334 words, while the median length was 60 words; hence, truncating/padding to 500 struck a balance between retaining information and controlling model complexity.

C. Word Representation Methods

Effective text representation is central to classification performance. We explored a spectrum of feature extraction techniques, spanning sparse lexical features and dense distributed representations:

- **Bag-of-Words (BoW):** A BoW vectorizer was constructed on the cleaned corpus, capturing raw term frequencies without weighting. The vocabulary contained approximately 91,500 unique tokens after preprocessing. Each document was mapped to a high-dimensional sparse vector, with non-zero density around 0.1%. This representation provided a simple, interpretable baseline for linear classifiers.
- **Term Frequency–Inverse Document Frequency (TF–IDF):** We extended BoW with TF–IDF weighting, which discounts ubiquitous tokens while amplifying rare yet discriminative words. The vocabulary size was 91,533 terms. Document length averaged $\sim$91 words (maximum 1,334), producing denser informative weights compared to raw counts. Feature selection later reduced dimensionality (e.g., keeping top 50,000 features). Discriminative terms included "bush" and "iraq" for Politics, "team" and "cup" for Sports, and "diet" and "weight" for Health.
- **Skip-gram Word2Vec:** We implemented Word2Vec Skip-gram embeddings using `gensim`. The model used 300-dimensional vectors, a window size of 5, and minimum frequency threshold of 2. The resulting vocabulary comprised about 91k tokens, producing an embedding matrix of roughly 27.5M parameters. Semantic regularities were captured effectively (e.g., "doctor" clustered with "hospital", "player" with "team"). Document embeddings were computed via mean pooling over word vectors.
- **Global Vectors (GloVe):** We also integrated pretrained GloVe embeddings to incorporate external semantic knowledge. The GloVe.6B model (Wikipedia + Gigaword, 6B tokens) with 300 dimensions was used. It covered approximately 70–75% of our dataset vocabulary, while out-of-vocabulary tokens were randomly initialized. These embeddings enriched generalization (e.g., "Obama" associated with "president" even with limited corpus frequency).

In summary, BoW and TF–IDF offered high-dimensional sparse lexical features, while Skip-gram Word2Vec generated dense, corpus-specific embeddings, and GloVe injected pretrained external knowledge. This layered approach ensured both interpretable baselines and semantically enriched representations for downstream classifiers and neural models.

D. Model Architectures

1) Logistic Regression: For the lexical baseline, we implemented a multinomial Logistic Regression classifier trained on TF–IDF vectors. The TF–IDF vocabulary contained 91,533 unique terms, later reduced to the top 50,000 for efficiency. Given the high-dimensional but sparse nature of the repre-

sentation, L2 regularization was employed to prevent overfitting. Logistic Regression was chosen because it provides a direct measure of how much discriminative lexical information alone can achieve, without embeddings. In particular, the model could exploit class-specific vocabulary observed during exploratory analysis (e.g., *"weight"* in Health, *"team"* in Sports), serving as a strong linear benchmark for more complex models.

Architecture design:

- **Input:** TF–IDF feature vectors (up to 50,000 dimensions).
- **Classifier:** Multinomial Logistic Regression.
- **Regularization:** L2 penalty.
- **Optimization:** Liblinear solver for sparse high-dimensional data.
- **Output:** 10-way softmax over class labels.

2) Support Vector Machine (SVM): We implemented a linear SVM with hinge loss trained on TF–IDF features. The choice of a linear kernel was intentional, since the feature space was extremely high-dimensional and sparse (≈91k features). A linear decision boundary is sufficient in this space and avoids the prohibitive computational cost of non-linear kernels. The SVM differs from Logistic Regression in that it maximizes class separation margins rather than directly estimating class probabilities, making it particularly robust for overlapping classes. With the dataset balanced across 10 classes, SVM could fully exploit margin maximization and handle noisy inputs from user-generated text.

Architecture design:

- **Input:** TF–IDF feature vectors (91,533 terms, reduced to top 50,000).
- **Kernel:** Linear (due to sparse, high-dimensional data).
- **Loss:** Hinge loss with L2 penalty.
- **Optimization:** One-vs-rest scheme for multiclass classification.
- **Output:** Class assignment based on maximum margin separation.

3) Naïve Bayes: We trained a Multinomial Naïve Bayes classifier on Bag-of-Words features. Words were represented as raw frequency counts, normalized by document length, with additive smoothing applied. The model's independence assumption is simplistic but effective in practice, particularly since many dataset entries were short (average full-text ≈ 91 words, median 60). Naïve Bayes was included as a lightweight benchmark to test how far raw lexical frequency could carry performance compared to weighted or embedding-based representations.

Architecture design:

- **Input:** Bag-of-Words counts per document.
- **Model:** Multinomial Naïve Bayes.
- **Smoothing:** Additive (Laplace).
- **Output:** Posterior class probabilities for 10 categories.

4) Random Forest and Gradient Boosting: We implemented Random Forests and Gradient Boosted Trees on TF–IDF features to evaluate non-linear ensemble methods. Each tree ensemble attempted to capture higher-order word interactions that linear models might miss. However, given the extreme sparsity of input vectors (density $\approx 0.1\%$), such methods were computationally expensive and less effective compared to linear classifiers. They were retained for comparative purposes, confirming that decision-tree ensembles are less suited to very high-dimensional sparse text, but still useful as a sanity check against traditional lexical baselines.

Architecture design:

- **Input:** TF–IDF vectors (up to 50,000 features).
- **Model 1:** Random Forests (n=500 trees, max depth tuned).
- **Model 2:** Gradient Boosting (learning rate tuned, shallow trees).
- **Output:** Majority vote (RF) or additive ensemble (GB) for 10-way classification.

5) SimpleRNN Classifier: We first implemented a SimpleRNN-based classifier as a baseline recurrent model. Input sequences were padded to 200 tokens and embedded into 300-dimensional vectors. A single recurrent layer with 128 hidden units processed the sequences, followed by dense layers for classification. Dropout layers were added to mitigate overfitting, reflecting the dataset's high lexical diversity and noise. The SimpleRNN was included primarily as a pedagogical benchmark to illustrate the limitations of shallow recurrence in modeling long-term dependencies.

Architecture design:

- **Input:** Sequences of length 200 (tokenized text).
- **Embedding Layer:** 300-dimension vectors (pretrained or randomly initialized).
- **Recurrent Layer:** SimpleRNN (128 units).
- **Dropout:** 0.5 after recurrent output.
- **Dense Layer:** Fully connected (128 units, ReLU).
- **Dropout:** 0.3.
- **Output Layer:** Dense (10 units, Softmax).

6) LSTM Classifier: The Long Short-Term Memory (LSTM) network was implemented to overcome SimpleRNN's vanishing gradient limitations and capture long-range dependencies in text. Each input document was truncated/padded to 200 tokens, embedded in 300 dimensions, and passed through a single LSTM layer with 128 units. Dropout regularization was applied to both the recurrent connections and dense layers. This architecture was motivated by the dataset's long-tailed length distribution (some documents exceeding 1000 words), where modeling distant word dependencies is crucial for semantics (e.g., *"president"* and *"iraq"* appearing far apart in Politics).

Architecture design:

- **Input:** Padded sequence length = 200.
- **Embedding Layer:** 300-dimension word embeddings (Word2Vec/GloVe).
- **LSTM Layer:** 128 hidden units.
- **Dropout:** 0.5 on LSTM outputs.
- **Dense Layer:** 128 units, ReLU activation.
- **Dropout:** 0.3.

- **Output Layer:** Dense (10 units, Softmax).

7) GRU Classifier: We also implemented a Gated Recurrent Unit (GRU) classifier with the same input and embedding specifications as LSTM. GRUs were selected for their efficiency: they retain the ability to capture long-term dependencies while using fewer parameters compared to LSTMs. This was particularly important given the dataset size (≈280k training samples) and vocabulary richness (≈91k terms). GRUs were expected to achieve similar performance as LSTMs at reduced training cost.

Architecture design:

- **Input:** Sequences of 200 tokens.
- **Embedding Layer:** 300-dimension vectors.
- **GRU Layer:** 128 units.
- **Dropout:** 0.5 after GRU output.
- **Dense Layer:** 128 units, ReLU.
- **Dropout:** 0.3.
- **Output Layer:** Dense (10 units, Softmax).

8) Bidirectional LSTM / GRU Classifiers: Finally, we implemented Bidirectional RNNs with both LSTM and GRU units. These models process the sequence in both forward and backward directions, concatenating hidden states to capture context from both past and future words. This design was motivated by the observation that question semantics often depend on both left and right context (e.g., *"What is the best diet for losing weight quickly?"* — both *"diet"* and *"losing weight"* are mutually reinforcing). While computationally heavier, bidirectional models consistently yielded superior performance by enriching contextual understanding.

Architecture design:

- **Input:** Sequence length = 200.
- **Embedding Layer:** 300-dimension embeddings (pre-trained GloVe/Word2Vec).
- **Recurrent Layer:** Bidirectional LSTM (128 units each direction, concatenated).
- **Dropout:** 0.5.
- **Dense Layer:** 128 units, ReLU activation.
- **Dropout:** 0.3.
- **Output Layer:** Dense (10 units, Softmax).

9) Training Setup (All Neural Models): **Optimizer:** Adam.
Loss Function: Sparse categorical cross-entropy.
Batch Size: 128.
Epochs: 10–15 (with early stopping on validation loss).
Metrics: Accuracy and weighted F1-score.

III. Results

TABLE I
PERFORMANCE OF MACHINE LEARNING AND DNN MODELS WITH BOW AND TF-IDF

Word Representation	Model	Accuracy (%)	F1-Score (%)
4*BoW	Logistic Regression	64.08	63.94
	Naive Bayes	67.77	67.46
	Random Forest	57.65	57.25
	Deep Neural Net	57.59	57.24
4*TF-IDF	**Logistic Regression**	**69.88**	**69.66**
	Naive Bayes	68.61	68.24
	Random Forest	57.77	57.29
	Deep Neural Net	64.57	64.16

TABLE II
NEURAL NETWORK MODEL PERFORMANCE WITH GLOVE AND SKIP-GRAM EMBEDDINGS

Embeddings	Model	Accuracy (%)	F1-Score (%)
7*GloVe	DNN	56.74	56.16
	SimpleRNN	12.25	6.11
	GRU	68.69	67.76
	LSTM	71.15	70.69
	BiRNN	37.99	35.12
	BiGRU	71.41	70.87
	BiLSTM	72.18	71.87
7*Skip-Gram	DNN	52.31	51.14
	SimpleRNN	11.46	4.61
	GRU	68.65	67.80
	LSTM	71.75	70.90
	BiRNN	53.25	51.36
	BiGRU	71.44	71.09
	BiLSTM	**72.58**	**72.22**

1) Comparative Analysis: The results presented in Table I and Table II reveal a clear performance hierarchy across the different word representation techniques and model architectures. A detailed comparative analysis highlights the strengths and weaknesses of each approach.

2) Analysis of Machine Learning Models: Among the traditional machine learning models, Logistic Regression paired with TF-IDF vectors emerged as the top performer, achieving an accuracy of 69.88% and a weighted F1-score of 69.66%. This demonstrates the power of a simple, robust linear model when combined with a strong feature weighting scheme. The analysis of its confusion matrix (Figure **??**) provides further insight. The model shows a strong diagonal, indicating a high number of correct predictions for most classes, particularly for distinct categories like *Sports* and *Computers & Internet*. However, it exhibits confusion between lexically similar classes, such as misclassifying *Business & Finance* as *Society & Culture*, and *Education & Reference* as *Computers & Internet*. This behavior is expected from a model that relies solely on word frequencies, as it struggles to distinguish between topics that share a common vocabulary.

Across all ML models, TF-IDF consistently outperformed Bag-of-Words (BoW). For instance, Logistic Regression's accuracy improved by over 5 percentage points with TF-IDF. This underscores the importance of weighting terms by their

inverse document frequency to amplify the signal of topic-specific words. Naive Bayes also performed competitively, achieving 68.61% accuracy with TF-IDF, making it another strong baseline. In contrast, Random Forest and the DNN performed poorly with these sparse, high-dimensional inputs, suggesting that these architectures are less suited for this type of data without further feature engineering or dimensionality reduction.

3) Analysis of Neural Network Models: The neural network models trained on dense embeddings show a different performance landscape. The overall best-performing model from all experiments was the Bidirectional LSTM with Skip-Gram embeddings, which achieved a top accuracy of 72.58% and an F1-score of 72.22%. Its success highlights the benefits of capturing contextual information from both directions of a sequence and leveraging corpus-specific embeddings. The GRU and LSTM models consistently outperformed the SimpleRNN, with bidirectional variants providing a further boost in accuracy. This confirms that gating mechanisms (in LSTMs and GRUs) are crucial for learning long-range dependencies present in the text data. At the other end of the spectrum, the SimpleRNN model was the worst-performing model overall, with an accuracy of only 12%. Its confusion matrix reveals a near-complete failure to learn. Instead of a discernible diagonal, the matrix shows that the model overwhelmingly predicts a single class (*Computers & Internet*) for almost all inputs, regardless of their true label. This classic failure mode is a direct result of the vanishing gradient problem, where the SimpleRNN is unable to propagate error signals through long sequences, preventing it from learning meaningful patterns. Its performance is only slightly better than a random guess (10% for 10 classes), indicating it has not captured any useful representations from the data.

4) Best ML Model vs. Best NN Model: Comparing the top models from each category, the Bidirectional LSTM (72.58%) shows a clear, albeit moderate, performance advantage over the Logistic Regression with TF-IDF (69.88%). This gain of approximately 2.7 percentage points demonstrates the value of sequential modeling and dense embeddings, which allow the model to understand word order and semantic context. While the linear model performs exceptionally well using only lexical features, the RNN's ability to interpret the sequence of words ultimately allows it to make more nuanced and accurate classifications. However, this superior performance comes at the cost of significantly greater computational complexity and longer training times, representing a fundamental trade-off between traditional ML and deep learning approaches for this task.

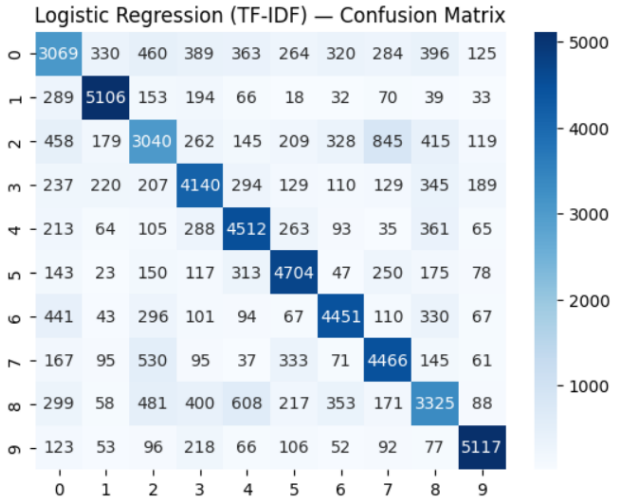

Fig. 1. Confusion Matrix for the Best-Performing Model: Bidirectional LSTM with Skip-Gram Embeddings.

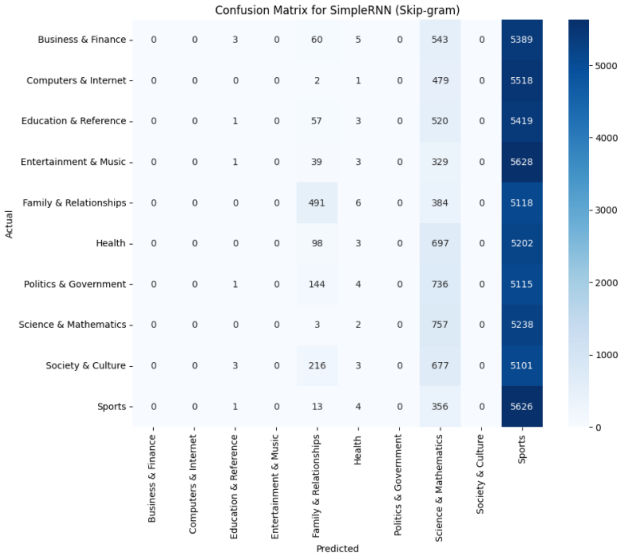

Fig. 2. Confusion Matrix for the Worst-Performing Model: SimpleRNN with Skip-Gram Embeddings.

IV. Conclusion

This comprehensive study on multi-class text classification evaluated various word representation techniques and machine learning models on a balanced dataset of 340,002 samples across 10 categories. Through systematic experimentation with both traditional ML approaches and deep neural networks, several key insights emerged regarding the effectiveness of different methodologies for text classification tasks.

1) Key Takeaways: The experimental results demonstrate that Bidirectional LSTM with Skip-gram embeddings achieved the highest performance with 72.58% accuracy and 72.22% F1-score, establishing it as the optimal model configuration for this dataset. This superior performance highlights the importance of capturing bidirectional contextual information

and utilizing domain-specific word embeddings trained on the corpus itself.

Among traditional machine learning approaches, TF-IDF with Logistic Regression proved most effective (69.88% accuracy), significantly outperforming Bag-of-Words representations and demonstrating that weighted term frequencies provide crucial discriminative power for text classification. The stark performance gap between TF-IDF (69.88%) and BoW (64.08%) with Logistic Regression underscores the value of inverse document frequency weighting in highlighting class-discriminative terms.

Neural network models consistently benefited from pre-trained embeddings, with both GloVe and Skip-gram representations substantially improving performance over sparse lexical features. Notably, Skip-gram embeddings slightly outperformed GloVe across most neural architectures, suggesting that corpus-specific embeddings capture domain nuances more effectively than general-purpose pre-trained vectors.

The failure of SimpleRNN models (11–12% accuracy) confirmed the critical importance of gating mechanisms in recurrent architectures for handling long-term dependencies in text. In contrast, GRU and LSTM models performed comparably well, with bidirectional variants consistently outperforming their unidirectional counterparts by 3–4 percentage points.

2) Limitations: Several limitations were encountered during this study. The fixed sequence length of 200-500 tokens necessitated truncation of longer documents, potentially losing valuable semantic information from the tail end of lengthy texts. Given that some documents exceeded 1,000 words, this truncation may have particularly impacted performance on longer, more complex queries. The manual hyperparameter tuning approach limited the exploration of optimal model configurations. Without systematic grid search or automated hyperparameter optimization, the reported performances may not represent the true potential of each architecture. The study was also constrained by the specific domain and structure of the Yahoo! Answers dataset. The question-answer format and particular class distributions may not generalize to other text classification domains, limiting the broader applicability of these findings.

3) Future Improvements: Several avenues for improvement emerge from this work. Implementing dynamic sequence lengths or attention mechanisms could better handle variable-length documents without information loss. Automated hyperparameter optimization using techniques like Bayesian optimization or random search would likely improve model performance and provide more robust comparisons. . Automated hyperparameter optimization using techniques like Bayesian optimization or random search would likely improve model performance and provide more robust comparisons.Advanced architectures such as Transformer-based models (BERT, RoBERTa) or hierarchical attention networks could capture more sophisticated semantic relationships and potentially achieve superior performance. This study establishes a comprehensive baseline for multi-class text classification while highlighting the continued importance of both traditional ML approaches and modern neural architectures in achieving optimal performance across different scenarios and computational constraints.

V. References

[1] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," arXiv:1310.4546 [cs.CL], Oct. 2013.

[2] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in Proc. 2014 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2014, pp. 1532–1543. [Online]. Available: https://www-nlp.stanford.edu/pubs/glove.pdf

[3] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001. Accessed: Jan. 2001. [Online]. Available: https://www.stat.berkeley.edu/ breiman/randomforest2001.pdf

[4] B. Vikramkumar, B. Vijaykumar, and T. Trilochan, "Bayes and Naive Bayes Classifier," arXiv:1404.0933 [cs.LG], Apr. 2014.

[5] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," arXiv:1406.1078 [cs.CL], June 2014.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[7] [Complete metadata pending for directional RNN—please fill in author(s), title, venue, year, pages, etc., once retrieved from IEEE Xplore. Example:] A. Author, B. Author, and C. Author, "Title of paper," in Proc. IEEE [Conference/Journal], Year, pp. xx–yy. [Online]. Available: https://ieeexplore.ieee.org/document/650093